

Setup Kafka

Installing Kafka

Objectives

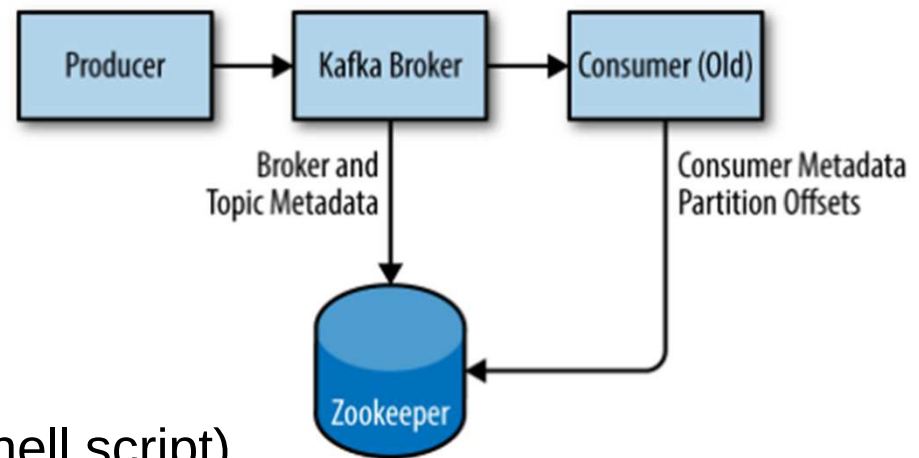
- Kafka Setup
- Practice
- Configuration Parameters
- Hardware Selection

Kafka Setup

- Products Required
 - Full Java Development Kit (JDK) – Kafka Built using Java
 - Apache Zookeeper – used by Kafka for storing metadata for the brokers, as well as consumer client details
 - Apache Kafka

- Steps for Installation

1. Install JDK (8.0)
2. Install Zookeeper
3. Install Kafka
4. Create & Verify a topic (shell script)
5. Produce a message (shell script)
6. Consume the message (shell script)



Install JDK

- We will use vagrant to manage virtual environments created by Virtual box
 - Install Virtual Box
 - Install Vagrant
 - Create Virtual Machine using Vagrant
 - `vagrant init ol7-latest`
 - `vagrant up`
 - Connect to Virtual environment using 'ssh' and install JDK
 - `vagrant ssh`
 - `cd /vagrant/software`
 - `sudo rpm -ivh /vagrant/software/jdk-8u51-linux-x64.rpm`
 - Verify if JDK is installed
 - `which java`
 - `java -version`

Install and Start Zookeeper

- Execute below commands using 'root' account:
 - `su` (change to root account)
 - `tar -zxf /vagrant/software/zookeeper-3.4.6.tar.gz`
 - `mv zookeeper-3.4.6 /usr/local/zookeeper`
 - `chown -R vagrant /usr/local/zookeeper`
 - `mkdir -p /var/lib/zookeeper`
 - `chown -R vagrant /var/lib/zookeeper`
 - `exit`
- Execute below commands from 'vagrant' user
 - `cat > /usr/local/zookeeper/conf/zoo.cfg << EOF`
 - `> tickTime=2000`
 - `> dataDir=/var/lib/zookeeper`
 - `> clientPort=2181`
 - `> EOF`

***tickTime** - the basic time unit in milliseconds used by ZooKeeper. It is used to do heartbeats and the minimum session timeout will be twice the tickTime.*

***dataDir** - the location to store the in-memory database snapshots and, unless specified otherwise, the transaction log of updates to the database.*

***clientPort** - the port to listen for client connections*
 - `export JAVA_HOME=/usr/java/jdk1.8.0_51`
 - `/usr/local/zookeeper/bin/zkServer.sh start`

Using Zookeeper

- Connect to Zookeeper
 - `%ZOOKEEPER_HOME%/bin/zkCli.sh -server 127.0.0.1:2181`
- Start by issuing list command
 - `ls /brokers/topics/test`
- Create a new znode and associates the string "my_data" with the node
 - `create /zk_test my_data`
- Verify if 'zk_test' is created
 - `ls /`
- Certify that the data was associated with the znode by running
 - `get /zk_test`
- We can change the data associated with zk_test by issuing the set command
 - `set /zk_test junk`
 - `ls /zk_test`
- Finally, let's delete the node by issuing and quit
 - `delete /zk_test`
 - `ls /`
 - `quit`
- Zookeeper in Production
 - You should run ZooKeeper in replicated mode.
 - A replicated group of servers in the same application is called a quorum, and in replicated mode, all servers in the quorum have copies of the same configuration file.

```
tickTime=2000
dataDir=/var/zookeeper
clientPort=2181
initLimit=5
syncLimit=2
server.1=zoo1:2888:3888
server.2=zoo2:2888:3888
server.3=zoo3:2888:3888
```

Install & Start Kafka

- Execute below commands with 'root' account
 - `su`
 - `tar -zxf /vagrant/software/kafka_2.11-0.9.0.1.tgz`
 - `mv kafka_2.11-0.9.0.1 /usr/local/kafka`
 - `chown -R vagrant /usr/local/kafka`
 - `mkdir /tmp/kafka-logs`
 - `chown -R vagrant /tmp/kafka-logs`
 - `exit`
- Execute below commands using 'vagrant' user
 - `export JAVA_HOME=/usr/java/jdk1.8.0_51`
 - `/usr/local/kafka/bin/kafka-server-start.sh -daemon /usr/local/kafka/config/server.properties`

Create a topic and test it

- Create and Verify a topic

- `/usr/local/kafka/bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic test`
- `/usr/local/kafka/bin/kafka-topics.sh --zookeeper localhost:2181 --describe --topic test`

- Produce the message

- `/usr/local/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic test`
- Type the message and press Ctrl+D

- Consume the message

- `/usr/local/kafka/bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic test --from-beginning`
- Press Ctrl+C

General Broker - Configuration Parameters

- Kafka distribution is sufficient to run a standalone server as a proof of concept, many options can be left to default settings.
- Below parameters deal with basic configuration, most them must be changed to run properly in cluster with other brokers.
 - **broker.id** – Unique integer identifier, good choice is to use host name if has unique integer (eg. host1.example.com)
 - **port** - Kafka with a listener on TCP port 9092
 - **zookeeper.connect** – The location of the Zookeeper used for storing the broker metadata
 - hostname , the hostname or IP address of the Zookeeper server.
 - port , the client port number for the server.
 - /path , an optional Zookeeper path to use as a chroot environment for the Kafka cluster. If it is omitted, the root path is used.
 - **log.dirs** – Kafka persists all messages to disk, and these log segments are stored in the directories specified in here.
 - **num.recovery.threads.per.data.dir** – Kafka uses a configurable pool of threads for handling log segments
 - **auto.create.topics.enable** – Kafka configuration specifies that the broker should automatically create a topic under the following circumstances
 - When a producer starts writing messages to the topic
 - When a consumer starts reading messages from the topic
 - When any client requests metadata for the topic

Topic Defaults - Configuration Parameters

- `num.partitions`
 - How many partitions a new topic is created with, primarily when automatic topic creation is enabled (default setting)
 - Default is one partition
 - Number of partitions can only be increased, but not decreased
- How long Kafka will retain messages is by time achieved using `log.retention.ms`, `log.retention.minutes`, `log.retention.hours`, `log.retention.bytes`
 - Default is 168 hours
 - When multiple configurations are present, then smaller unit size takes precedence.
 - `log.retention.bytes` – Another way to expire messages is based on the total number of bytes of messages retained
 - Applied per partition, This means that if you have a topic with 8 partitions, and `log.retention.bytes` is set to 1 GB, the amount of data retained for the topic will be 8 GB at most.
- Log Segment Configuration – Way to control when log segments are closed.
 - As messages are produced to the Kafka broker, they are appended to the current log segment for the partition. Once the log segment has reached the size specified by the `log.segment.bytes` parameter, which defaults to 1 GB, the log segment is closed and a new one is opened.
 - `log.segment.bytes`
 - `log.segment.ms`
- `message.max.bytes` – maximum size of a message that can be produced (default is 1MB)

Hardware Selection

- Selecting an appropriate hardware configuration for a Kafka broker can be more art than science.
 - Disk Throughput – SSDs have drastically lower seek and access times and will provide the best performance. HDDs, are more economical and provide more capacity per unit.
 - Disk Capacity
 - Memory – Having more memory available to the system for page cache will improve the performance of consumer clients. Kafka itself does not need much heap memory configured for the Java Virtual Machine (JVM).
 - Networking
 - CPU
- Kafka in the Cloud
 - A common installation for Kafka is within cloud computing environments
 - Event Hub – https://cloud.oracle.com/en_US/event-hub
 - Power Of Kafka – https://www.youtube.com/watch?v=J_eErs9nQNs

Summary

- Kafka Setup
- Practice
- Configuration Parameters
- Hardware Selection

Practice

- This practice teaches